PLAGIARISM DETECTOR: AN NLP-BASED TEXT SIMILARITY DETECTION SYSTEM

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

Submitted by

Name	Roll No.
Khan Naushad	231227
Jamil Khatri	231231
Sakharkar Shaan	231244

Project Guide

Prof. Farhana Siddiqui



DEPARTMENT OF COMPUTER ENGINEERING
M. H. SABOO SIDDIK COLLEGE OF ENGINEERING, MUMBAI
UNIVERSITY OF MUMBAI

(A.Y. 2026–2027)

ACKNOWLEDGMENT

We express our sincere gratitude to everyone who supported us during the development of our mini-project, “Plagiarism Detector: An NLP-Based Text Similarity Detection System.”

We thank our project guide for valuable guidance, suggestions, and encouragement. This support helped us understand Natural Language Processing, n-gram features, TF-IDF vectorization, and cosine similarity, and apply these concepts in a practical application.

We also thank the faculty members and the Department of Computer Engineering at M. H. Saboo Siddik College of Engineering for the academic support and learning environment provided during this project.

Finally, we are grateful to our friends and families for their patience, motivation, and support throughout the project work.

We sincerely thank all those who contributed directly or indirectly to the completion of this project.

Table of Contents

Sr. No.	Title	Page No.
1.	Introduction	4
2.	Methodology	5
	2.1 Text Preprocessing and Tokenization	5
	2.2 Algorithm	5–6
	2.3 N-Gram Extraction and TF-IDF Vectorization	7
	2.4 Cosine Similarity and Scoring Metrics	7
	2.5 Sentence-Level Matching and Highlighting	7
	2.6 Vectorizer Initialization and Caching	8
	2.7 User Interface and Output Display	8
3.	Implementation	9
	3.1 System Architecture	9
	3.2 Tools and Libraries Used	9
	3.3 Feature Breakdown	10
4.	Code and Output	11–13
5.	Conclusion	14
	References	14

List of Figures

Figure name	Pg. No.
2.1 Plagiarism Detection Algorithm using TF-IDF and Cosine Similarity	6
3.1 System Architecture of the Plagiarism Detector	9
4.1 Illustrative user interface mock-up	12
4.2 Illustrative similarity report mock-up	13

1. INTRODUCTION

Academic institutions, research journals, and digital publishing platforms receive large volumes of assignments, articles, and technical reports. Maintaining originality is important, but manually comparing each submission with source material requires time and effort. A preliminary text-comparison tool can help reviewers locate passages that deserve closer inspection.

In today's digital world, a large amount of information is shared through academic papers, research articles, assignments, project reports, and online publications. Maintaining originality in written content is essential for academic integrity and ethical research. Plagiarism occurs when a person uses another person's words, ideas, or work without appropriate acknowledgment. Manually checking documents for copied content is a time-consuming process, especially when the amount of digital information is continuously increasing. Therefore, an automated text similarity detection system can help identify potentially matching content efficiently.

Traditional methods of plagiarism detection mainly depend on exact word or sentence matching. These methods may fail to identify similarities when the text is modified through changes in punctuation, capitalization, or word arrangement. Natural Language Processing (NLP) provides techniques to process and analyze textual information and identify similarities between documents. By applying text preprocessing, tokenization, n-gram extraction, and statistical text representation, a system can compare documents more systematically.

The Plagiarism Detector: An NLP-Based Text Similarity Detection System is designed to compare a reference document with a submitted document and identify potentially similar passages. The system uses text preprocessing to convert the input into a consistent format and applies unigram and bigram extraction to identify individual words and consecutive word pairs. Term Frequency–Inverse Document Frequency (TF-IDF) is then used to convert the text into numerical feature vectors. Cosine similarity measures the similarity between these vectors and produces a score representing the degree of textual overlap.

The proposed system also performs sentence-level comparison to identify submitted sentences that closely match sentences in the reference text. A configurable similarity threshold is used to flag candidate matching passages. The results are displayed through a user-friendly web interface developed using Python and Streamlit, with Scikit-learn for TF-IDF vectorization and cosine similarity and NLTK for sentence tokenization.

The main objective of this project is to reduce the manual effort involved in preliminary document comparison and demonstrate the practical application of NLP techniques in text analysis. The system provides document similarity scores, estimated matched-text percentages, and highlighted candidate passages to support further examination.

However, text similarity does not automatically prove plagiarism. Common expressions, properly cited quotations, and legitimate reuse may also produce matching results. Therefore, the generated report should be treated as a preliminary screening tool, and the final assessment should be made through careful human review.

Project Overview

The Plagiarism Detector is an NLP-based web application for preliminary comparison of a reference document and a submitted document. It normalizes text, creates unigram and bigram TF-IDF vectors, calculates document and sentence cosine similarities, highlights candidate matching passages, and presents a report through an interactive interface.

2. METHODOLOGY

2.1 Text Preprocessing and Tokenization

2.1.1 Vector Space Representation Model

The system uses a statistical Vector Space Model based on TF-IDF and cosine similarity. Each input is represented as a numerical vector whose dimensions correspond to terms and adjacent word pairs. TF-IDF gives greater weight to terms that are frequent in a document but less common across the compared corpus. Including unigrams and bigrams captures individual word overlap and some local phrase structure.

This approach does not require a large training dataset or GPU. The vectorizer learns its vocabulary from the text supplied for each comparison, making the implementation compact and explainable. Its limitation is that lexical overlap is emphasized; semantic equivalence between different wording is not directly modeled.

2.1.2 Input Text Processing

The user supplies a reference text and a submitted text. The application checks that both fields contain non-whitespace characters before analysis. Sentence segmentation is performed first for local matching; normalized text is then used for vectorization.

Processing sequence:
(i) Accept reference and submitted text.
(ii) Validate that both inputs are non-empty.
(iii) Split each text into sentences.
(iv) Convert text to lowercase and remove punctuation/special characters.
(v) Tokenize words and remove English stop words where appropriate.
(vi) Extract unigram and bigram TF-IDF features.
(vii) Calculate document-level and sentence-level cosine similarities.
(viii) Apply a threshold, identify candidate matches, and display results.

2.2 Algorithm

2.2.1 TF-IDF and Cosine Similarity

1. Input: Accept the reference text and submitted text from the user.
2. Preprocessing: Convert the text to lowercase, remove punctuation and extra spaces, and split the text into sentences.
3. Tokenization: Divide the text into individual words and prepare the text for feature extraction.
4. N-Gram Extraction: Extract unigrams (single words) and bigrams (two consecutive words).
5. TF-IDF Vectorization: Convert both documents into numerical vectors by calculating the TF-IDF weights of their terms.
6. Document Similarity: Calculate the cosine similarity between the reference and submitted document vectors.
7. Sentence-Level Comparison: Compare each submitted sentence with the reference sentences using cosine similarity.
8. Threshold Checking: Compare sentence similarity scores with a predefined threshold to identify potentially matching passages.
9. Result Generation: Calculate the estimated matched-text percentage and highlight candidate matching sentences.
10. Output: Display the overall similarity score, matched-text percentage, and highlighted passages to the user..

The flow below summarizes validation, preprocessing, vectorization, comparison, and report generation.

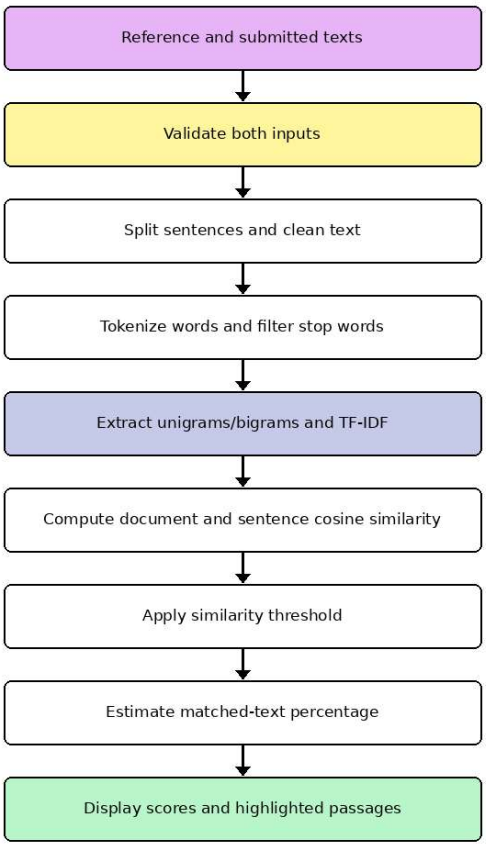


Fig. 2.1: Plagiarism Detection Algorithm using TF-IDF and Cosine Similarity

Algorithm steps:

1. Start the application and initialize NLP resources.
2. Accept the reference and submitted texts.
3. If either input is empty, display a warning and stop analysis.
4. Split texts into sentences and normalize the text.
5. Fit a TF-IDF vectorizer using unigram and bigram features.
6. Calculate overall document cosine similarity.
7. Calculate each submitted sentence's maximum similarity against reference sentences.
8. Mark sentences above the selected threshold and estimate the proportion of submitted words in matched sentences.
9. Display the score, matches, and highlighted passages.

The output is a screening signal only; a reviewer must inspect highlighted passages and the source context before making any conclusion.

2.3 N-Gram Extraction and TF-IDF Vectorization

An n-gram is a contiguous sequence of n tokens. A unigram contains one word, while a bigram contains two adjacent words. The vectorizer uses ngram_range=(1, 2) to capture individual terms and short phrases.

A smoothed inverse-document-frequency form used by Scikit-learn is:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times [\log((1 + N)/(1 + \text{df}(t))) + 1]$$

Here, N is the number of documents in the fitted collection and df(t) is the number of documents containing term t. Scikit-learn normalizes the resulting vectors by default, supporting cosine-based comparison.

2.4 Cosine Similarity and Scoring Metrics

Cosine similarity measures the normalized dot product of vectors A and B:

$$\text{cosine}(A,B) = (A \cdot B) / (||A|| \times ||B||)$$

For non-negative TF-IDF vectors, scores generally range from 0 to 1. A value near 1 indicates strong lexical overlap in the fitted feature space, while a value near 0 indicates little shared weighted vocabulary. Document similarity is the cosine score for the two complete text vectors.

2.5 Sentence-Level Matching and Highlighting

Each submitted sentence is compared with each reference sentence. The maximum reference-sentence score is retained for each submitted sentence. If that score meets or exceeds the selected threshold, the sentence is treated as a candidate match and highlighted. The matched-text percentage is estimated as words in marked submitted sentences divided by the total submitted word count, multiplied by 100.

This percentage depends on the threshold and sentence segmentation. It is not a universal plagiarism percentage and may count a whole sentence even when only part of it overlaps.

2.6 Vectorizer Initialization and Caching

The TF-IDF vectorizer is created for each analysis so its vocabulary and inverse-document-frequency values correspond to the pair of texts being compared. In Streamlit, stable resources such as the NLTK stop-word list can be cached to avoid repeated loading. A text-specific vectorizer should not be reused across unrelated comparisons unless that behavior is intended.

2.7 User Interface and Output Display

The interface provides separate input areas for reference and submitted text, a similarity-threshold control, and an Analyze Similarity action. After validation and analysis, it displays overall cosine similarity, estimated matched-text percentage, and a sentence-level list of candidate matches with their scores.

2.8 Overall Methodology

The workflow is: input collection → validation → sentence segmentation and normalization → unigram/bigram TF-IDF vectorization → document cosine similarity → sentence-level comparison → thresholding → highlighted report. Each stage is modular so future versions can replace the vectorizer, add source-document retrieval, or include semantic embeddings.

3. IMPLEMENTATION

3.1 System Architecture

The system architecture of the Plagiarism Detector consists of several modules that work together to identify textual similarities between a reference document and a submitted document. First, the user enters both texts through the Streamlit web interface. The input validation module checks whether both text fields contain valid input. Next, the text preprocessing module converts the text into lowercase, removes punctuation and extra spaces, and divides the content into individual sentences using NLTK. The processed text is then passed to the TF-IDF vectorization module, which extracts unigram and bigram features and converts them into numerical vectors using Scikit-learn. The cosine similarity module calculates the overall document similarity and compares individual submitted sentences with reference sentences. A configurable threshold is applied to identify potential matching passages. Finally, the result module displays the document similarity score, estimated matched-text percentage, and highlighted candidate sentences through the Streamlit interface. This architecture provides an efficient and understandable approach to preliminary text similarity analysis.

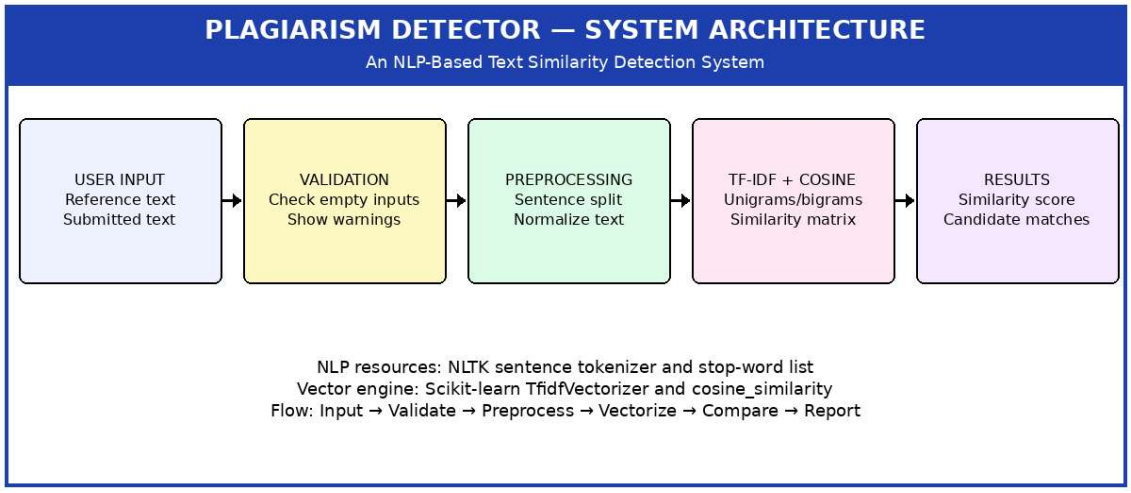


Fig. 3.1: System Architecture of the Plagiarism Detector

3.2 Tools and Libraries Used

Tool / Library	Purpose
Python	Core application and text processing
Streamlit	Interactive web interface
Scikit-learn	TF-IDF vectorization and cosine similarity
NLTK	Sentence tokenization and stop-word resources
re (regular expressions)	Text normalization and punctuation handling
Pillow	Report figure and mock-up generation
python-docx	Creation of editable Word report

3.3 Plagiarism Detector — Feature Breakdown

1. Text Input

Accepts a reference text and submitted text through separate input areas.

2. Input Validation

Checks for empty or whitespace-only fields and warns the user before analysis.

3. Preprocessing

Normalizes capitalization and punctuation, splits sentences, and tokenizes text.

4. N-Gram Features

Uses unigrams and bigrams to represent words and short phrases.

5. TF-IDF Vectorization

Transforms the texts into weighted numerical vectors.

6. Document Similarity

Calculates cosine similarity between full reference and submitted documents.

7. Sentence Matching

Compares each submitted sentence with reference sentences and retains the strongest match.

8. Threshold Control

Lets the reviewer adjust the cutoff for flagging candidate matching sentences.

9. Highlighting

Marks candidate matches so the reviewer can inspect them efficiently.

10. Report Display

Shows document similarity, estimated matched-text percentage, and sentence-level scores.

11. Explainability

Uses interpretable lexical features rather than claiming a black-box plagiarism verdict.

12. Limitations

Cannot prove intent, find uncited sources it has not been given, or reliably detect semantic paraphrases.

4. CODE AND OUTPUT

4.1 Core Python Implementation (Part 1)

Core functions below normalize text and split it into sentences. Install the dependencies and NLTK resources before running the full application.

```
import re
import streamlit as st
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


@st.cache_resource
def load_stopwords():
    try:
        return set(stopwords.words('english'))
    except LookupError:
        nltk.download('stopwords', quiet=True)
        return set(stopwords.words('english'))


def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    return re.sub(r'\s+', ' ', text).strip()


def split_sentences(text):
    try:
        return [s.strip() for s in sent_tokenize(text) if s.strip()]
    except LookupError:
        nltk.download('punkt', quiet=True)
        try:
            nltk.download('punkt_tab', quiet=True)
        except Exception:
            pass
        return [s.strip() for s in sent_tokenize(text) if s.strip()]
```

4.2 Similarity Analysis and User Interface

The following implementation uses a shared sentence vectorizer for both sides, handles empty vocabularies, and reports sentence matches. It is an illustrative code listing; run it in a configured Python environment before claiming application execution.

```
def analyze(reference, submitted, threshold=0.40):
    ref_sents = split_sentences(reference)
    sub_sents = split_sentences(submitted)
    if not ref_sents or not sub_sents:
        return 0.0, 0.0, []


    vectorizer = TfidfVectorizer(
        preprocessor=clean_text,
        stop_words='english', ngram_range=(1, 2)
    )
    try:
        doc_matrix = vectorizer.fit_transform([reference, submitted])
        doc_score = float(cosine_similarity(
            doc_matrix[0:1], doc_matrix[1:2]
        )[0, 0])
    except ValueError:  # e.g. inputs contain no usable terms
        return 0.0, 0.0, []


    sentences = ref_sents + sub_sents
    sent_matrix = vectorizer.fit_transform(sentences)
    ref_matrix = sent_matrix[:len(ref_sents)]
    sub_matrix = sent_matrix[len(ref_sents):]
    scores = cosine_similarity(sub_matrix, ref_matrix).max(axis=1)
    matches = [(s, float(v)) for s, v in zip(sub_sents, scores)
               if v >= threshold]
    words = sum(len(s.split()) for s in sub_sents)
    matched_words = sum(len(s.split()) for s, _ in matches)
    pct = 100.0 * matched_words / words if words else 0.0
    return doc_score, pct, matches


st.title('Plagiarism Detector')
reference = st.text_area('Reference Text')
submitted = st.text_area('Submitted Text')
threshold = st.slider('Match threshold', 0.0, 1.0, 0.40, 0.05)
if st.button('Analyze Similarity'):
    if not reference.strip() or not submitted.strip():
        st.warning('Enter both texts before analysis.')
    else:
        score, matched_pct, matches = analyze(reference, submitted, threshold)
        st.metric('Document similarity', f'{score:.1%}')
        st.metric('Estimated matched-text share', f'{matched_pct:.1f}%')
        for sentence, value in matches:
            st.markdown(f'> **{value:.1%} match:** {sentence}')
```



Fig. 4.1: Illustrative interface mock-up (not a captured execution)

4.3 Illustrative Output and Interpretation

The following is a conceptual example of how results may be presented. It is not a result from a verified run and should be replaced with a real screenshot after executing the application.



Fig. 4.2: Illustrative similarity report mock-up (example values are not actual results)

Interpretation

A higher document-level score indicates greater weighted word/phrase overlap in the supplied texts. A sentence-level flag means that a sentence exceeded the configured threshold against at least one reference sentence. Reviewers should compare the highlighted context, check citations, and consider common terminology before deciding whether further action is needed.

5. CONCLUSION

The Plagiarism Detector: An NLP-Based Text Similarity Detection System is designed to identify potential similarities between reference and submitted documents using Natural Language Processing techniques. The project demonstrates how text preprocessing, tokenization, n-gram extraction, TF-IDF vectorization, and cosine similarity can be combined to develop an effective preliminary text comparison system.

The system first accepts text from the user and performs preprocessing to remove unnecessary punctuation, normalize capitalization, and divide the content into sentences. It then extracts unigram and bigram features to represent individual words and consecutive word pairs. TF-IDF assigns numerical weights to these features, while cosine similarity measures the degree of overlap between the reference and submitted texts. The system also performs sentence-level comparison to identify potentially matching passages and displays the results through an interactive web interface developed using Python and Streamlit.

The main advantage of this project is that it reduces the time and effort required for initial document comparison. The similarity score and highlighted passages help users identify sections that require further investigation. The use of Scikit-learn and NLTK makes the system lightweight, understandable, and suitable for demonstrating important NLP concepts without requiring extensive model training or specialized hardware.

However, the system has certain limitations. It compares the text provided by the user and does not automatically search the entire internet or a large collection of academic publications. It may identify legitimate quotations or common phrases as similar content and may fail to detect paraphrased text that uses different words. Therefore, the similarity score should not be considered direct proof of plagiarism. Human verification and examination of the original sources remain essential.

In the future, the system can be enhanced by integrating semantic similarity models, a searchable database of reference documents, multilingual text processing, citation detection, and advanced machine-learning techniques. These improvements could help identify more complex similarities and make the system more useful for academic institutions, researchers, and content creators.

REFERENCES

1. Scikit-learn Developers, “TfidfVectorizer Documentation.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

2. Scikit-learn Developers, “Cosine Similarity Documentation.” Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

3. NLTK Project, “Natural Language Toolkit Documentation.” Available at: https://www.nltk.org/

4. Streamlit, “Streamlit Documentation.” Available at: https://docs.streamlit.io/

5. Manning, C. D., Raghavan, P., and Schütze, H., Introduction to Information Retrieval. Cambridge University Press, 2008. Available at: https://nlp.stanford.edu/IR-book/

6. Bird, S., Klein, E., and Loper, E., Natural Language Processing with Python. O’Reilly Media, 2009. Available at: https://www.nltk.org/book/

7. Python Software Foundation, “Python Documentation.” Available at: https://docs.python.org/3/

8. Streamlit, “Build and Deploy Data Apps.” Available at: https://streamlit.io/